Système et Réseaux : TP5

février 2026 — Pierre Rousselin et Antoine Rozenknop

1 Problèmes à savoir traiter

Exercice 1 : Belmondo, de Funès et *mutex*

1. Télécharger et lire le programme `belmondo.c` sur l'ENT.
2. L'exécuter plusieurs fois : que constatez-vous ?
3. Modifier le programme de façon à ce que toutes les lignes du texte initial soient lues et que les acteurs ne parlent pas en même temps. Plutôt que d'obtenir un affichage de ce genre :

```
jJEj eTeE   TtIteENe S
ttiieennssT
U ME TIENS
ppaarrP A Rl lLaAa  B AbRbBaIaCrHrEbTbTiEi
cchheetDtEt NtOeUSe
DEUX
qquuiQiU I  rRrIiRiA
rraaT
A
ppeetPtEtTtTeEe
```

On veut quelque chose comme :

```
JE TE TIENS
ttuu  mmee  ttiieennss
PAR LA BARBICHETTE
llee  pprreemmiieerr
DE NOUS DEUX
qquuii  rriirraa
AURA UNE
ttaa
PETTE
```

Peu importe si un des acteurs enchaîne deux lignes avant que l'autre n'ait fini de traiter la sienne, ou si les lignes ne sont pas dans le bon ordre (on rectifiera au montage), l'essentiel est qu'elles y soient toutes.

--- * ---

Correction de l'exercice 1 : Dans les deux *threads* :

— Déplacer l'incrémentation de `data->i` au début. La section lecture/écriture sur `data->i` est critique et doit être verrouillée.

— La déclamation du texte est verrouillée.

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <string.h>

struct data {
        int i;
        pthread_mutex_t verrou_i;
        pthread_mutex_t verrou_affichage;
        char **texte;
        int *durees;
```

```
};

void *de_funes(void *arg);
void *belmondo(void *arg);

/* attendre delai microsecondes */
void microsleep(int delai);

int main(void)
{
        char *texte[] = {
                "je te tiens", "tu me tiens", "par la barbichette",
                "le premier", "de nous deux", "qui rira", "aura une",
                "ta", "pette", NULL, NULL };
        int durees[] = { 2, 2, 7, 1, 3, 2, 2, 1, 1, 0 };

        struct data donnees = { .i = 0, .texte = texte, .durees = durees };

        pthread_t acteur1, acteur2;

        pthread_mutex_init(&donnees.verrou_i, NULL);
        pthread_mutex_init(&donnees.verrou_affichage, NULL);

        /* rendre la sortie standard unbuffered */
        setvbuf(stdout, NULL, _IONBF, 0);

        if (pthread_create(&acteur1, NULL, de_funes, &donnees) != 0) {
                perror("pthread");
                return 1;
        }
        if (pthread_create(&acteur2, NULL, belmondo, &donnees) != 0) {
                perror("pthread");
                return 1;
        }
        pthread_join(acteur1, NULL);
        pthread_join(acteur2, NULL);
        pthread_mutex_destroy(&donnees.verrou_affichage);
        pthread_mutex_destroy(&donnees.verrou_i);

        return 0;
}

void *de_funes(void *arg)
{
        char *tirade = "";
        struct data *data = arg;
        char a_lire[100];
        int i, delai, num_ligne;
        while (tirade != NULL) {
                /* accès au texte d'origine */
                /* Section critique : lecture et écriture sur data->i */
                pthread_mutex_lock(&(data->verrou_i));
                num_ligne = data->i;
```

```
                ++(data->i);
                pthread_mutex_unlock(&(data->verrou_i));

                tirade = data->texte[num_ligne];
                if (tirade == NULL) {
                        break;
                }
                delai = data->durees[num_ligne] * .5e6 / strlen(tirade);

                /* préparation du texte à lire */
                for (i = 0; tirade[i] != 0; ++i) {
                        a_lire[i] = tirade[i] +
                                (tirade[i]==' ' ? 0 : 'A' - 'a');
                        microsleep(delai);
                }
                a_lire[i] = 0;

                /* lecture du texte */
                pthread_mutex_lock(&(data->verrou_affichage));
                for (i = 0; a_lire[i] != 0; ++i) {
                        putchar(a_lire[i]);
                        microsleep(delai);
                }
                putchar('\n');
                pthread_mutex_unlock(&(data->verrou_affichage));
        }
        return NULL;
}

void *belmondo(void *arg)
{
        char *tirade = "";
        struct data *data = arg;
        char a_lire[100];
        int i, delai, num_ligne;

        while (tirade != NULL) {
                /* accès au texte d'origine */
                pthread_mutex_lock(&(data->verrou_i));
                num_ligne = data->i;
                ++(data->i);
                pthread_mutex_unlock(&(data->verrou_i));
                tirade = data->texte[num_ligne];
                if (tirade == NULL) {
                        break;
                }
                delai = data->durees[num_ligne] * .5e6 / strlen(tirade);

                /* préparation du texte à lire */
                for (i = 0; tirade[i] != 0; ++i) {
                        a_lire[2 * i] = a_lire[2 * i + 1] = tirade[i];
                        microsleep(delai);
                }
```

```
                a_lire[2 * i] = 0;

                /* lecture du texte */
                pthread_mutex_lock(&(data->verrou_affichage));
                for (i = 0; a_lire[i] != 0; ++i) {
                        putchar(a_lire[i]);
                        microsleep(delai);
                }
                pthread_mutex_unlock(&(data->verrou_affichage));
                putchar('\n');
        }
        return NULL;
}

void microsleep(int delai)
{
        struct timespec ts = { .tv_sec = delai / 1000000,
                .tv_nsec = 1000 * (delai % 1000000)  };
        nanosleep(&ts, NULL);
}
```

--- * ---

Exercice 2 : Barrière (non réutilisable)

1. Télécharger sur l'ENT, lire, compiler et exécuter (plusieurs fois) le programme `barriere_non-sync.c`.
2. On voudrait que tous les chiffres soient affichés avant toutes les lettres (mais pas nécessairement dans l'ordre). Par exemple, ceci est acceptable pour $N = 5$:

   ```
   3
   4
   2
   1
   0
   A
   E
   B
   D
   C
   ```

   En utilisant variable(s) supplémentaire(s) et/ou sémaphores et/ou mutex, modifier le programme de façon à satisfaire la condition ci-dessus.

   Indication : lorsque plusieurs personnes se suivent et passent une porte, il arrive que chacun tienne la porte pour le suivant...

--- * ---

Correction de l'exercice 2 :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>
#define N 5
#define COEFF_ATTENTE 50000
```

```c
/* attendre delai microsecondes */
void microsleep(int delai);

int nb_threads_ok = 0;
pthread_mutex_t verrou_nb_th;
sem_t tous_finis;
void work(void);
void *chiffres_et_alphabet(void *);

int main(void)
{
        int i, nums[N];
        pthread_t th[N];
        srand(time(NULL));
        sem_init(&tous_finis, 0, 0);
        pthread_mutex_init(&verrou_nb_th, NULL);
        for (i = 0; i < N; ++i) {
                nums[i] = i;
                if (pthread_create(&th[i], NULL, chiffres_et_alphabet, &nums[i]) != 0) {
                        perror("create");
                        return 1;
                }
        }

        for (i = 0; i < N; ++i)
                pthread_join(th[i], NULL);
        sem_destroy(&tous_finis);
        pthread_mutex_destroy(&verrou_nb_th);
        return 0;
}

void *chiffres_et_alphabet(void *arg)
{
        int i = *((int *) arg);
        work();
        printf("%d\n", i);
        pthread_mutex_lock(&verrou_nb_th);
        if (++nb_threads_ok == N)
                sem_post(&tous_finis);
        pthread_mutex_unlock(&verrou_nb_th);
        sem_wait(&tous_finis);
        sem_post(&tous_finis);
        printf("%c\n", 'A' + i);
        return NULL;
}

void work(void) { microsleep((rand() % 10) * COEFF_ATTENTE); }

void microsleep(int delai)
{
        struct timespec ts = { .tv_sec = delai / 1000000,
                .tv_nsec = 1000 * (delai % 1000000)  };
        nanosleep(&ts, NULL);
}
```

--- * ---

Exercice 3 : Double-buffering

Le programme `space_invaders.c`, disponible sur moodle fait les choses suivantes `num_iter` fois :

— D'abord la prochaine image à afficher est calculée. Ici, simplement, toutes les lignes sont décalées et un pixel (représenté par `*`) est ajouté sur la première ligne.

— Ensuite, le terminal est effacé et l'image (les pixels de l'écran) est affichée.

1. Télécharger, lire, compiler et exécuter ce programme.
2. Votre mission maintenant est de rendre ce programme concurrent : un *thread* devrait être dédié au calcul du prochaine écran (on dit aussi de la prochaine *frame*) pendant qu'un autre affiche la *frame* courante.

   Vous aurez pour cela besoin de 2 écrans, un pour la `frame` en train d'être affichée (il sera lu par les deux *thread*) et un autre pour la prochaine (il sera modifié par un seul *thread*), en train d'être calculée. Pour éviter les copies intempestives, vous travaillerez également avec deux pointeurs globaux :

   ```
   int (*ecran_a_afficher)[HAUTEUR][LARGEUR];
   int (*ecran_a_modifier)[HAUTEUR][LARGEUR];
   ```

   chacun contenant l'adresse d'un des deux écrans. Un des *threads* se chargera d'échanger les valeurs des deux pointeurs au bon moment.
3. Il faudra bien sûr faire bien attention aux problèmes de synchronisation. Lancer le programme avec double-buffer avec la commande `time` devrait montrer qu'il prend (environ) deux fois moins de temps que la version sans double-buffer.

Pour plus d'informations sur cette technique répandue appelée *double-buffering*, voir `https://en.wikipedia.org/wiki/Multiple_buffering`

--- * ---

Correction de l'exercice 3 :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <semaphore.h>
#include <pthread.h>

#define HAUTEUR 40
#define LARGEUR 80
#define NUM_ITER_DFT 30
#define DELAI 250000

/* Attendre delai microsecondes */
void microsleep(int delai);

int ecran1[HAUTEUR][LARGEUR];
int ecran2[HAUTEUR][LARGEUR];
void effacer_lignes(int num_lignes);

int (*ecran_a_afficher)[HAUTEUR][LARGEUR];
int (*ecran_a_modifier)[HAUTEUR][LARGEUR];

void *afficher(void *num_iter);
void *modifier(void *num_iter);

sem_t modif_finie, ecrans_echanges;

int main(int argc, char *argv[])
{
```

```
        int num_iter = argc > 1 ? atoi(argv[1]) : NUM_ITER_DFT;
        pthread_t modifieur, afficheur;

        sem_init(&modif_finie, 0, 0);
        sem_init(&ecrans_echanges, 0, 1);

        ecran_a_modifier = &ecran1;
        ecran_a_afficher = &ecran2;

        if (pthread_create(&modifieur, NULL, modifier, &num_iter) != 0) {
                perror("create modifieur");
                return 1;
        }
        if (pthread_create(&afficheur, NULL, afficher, &num_iter) != 0) {
                perror("create afficheur");
                return 1;
        }

        pthread_join(modifieur, NULL);
        pthread_join(afficheur, NULL);
        sem_destroy(&ecrans_echanges);
        sem_destroy(&modif_finie);
        puts("YOU LOSE.\n");
        return 0;
}

void effacer_lignes(int num_lignes)
{
        printf("\033[2K");
        for(; num_lignes > 0; --num_lignes) {
                printf("\033[F");
                printf("\033[2K");
        }
}

void *modifier(void *num_iter)
{
        int n, nb_iter = *((int *) num_iter), i, j;
        for (n = 0; n < nb_iter; ++n) {
                sem_wait(&ecrans_echanges);
                /* calcul de l'affichage suivant */
                microsleep(DELAI); /* simuler un calcul long */
                for (i = HAUTEUR - 1; i > 0; --i)
                        for (j = 0; j < LARGEUR; ++j)
                                (*ecran_a_modifier)[i][j] = (*ecran_a_afficher)[i - 1][j];
                for (j = 0; j < LARGEUR; ++j)
                        (*ecran_a_modifier)[0][j] = 0;
                (*ecran_a_modifier)[0][rand() % LARGEUR] = 1;
                sem_post(&modif_finie);
        }
        return NULL;
}

void *afficher(void *num_iter)
{
        int n, nb_iter = *((int *) num_iter), i, j;
        int (*tmp)[HAUTEUR][LARGEUR];
```

```
        for (n = 0; n < nb_iter; ++n) {
                sem_wait(&modif_finie);

                /* échange des écrans */
                tmp = ecran_a_afficher;
                ecran_a_afficher = ecran_a_modifier;
                ecran_a_modifier = tmp;
                sem_post(&ecrans_echanges);

                microsleep(DELAI); /* simuler un affichage qui prend du temps */
                effacer_lignes(HAUTEUR);
                for (i = 0; i < HAUTEUR; ++i) {
                        for (j = 0; j < LARGEUR; ++j) {
                                if ((*ecran_a_afficher)[i][j] == 0)
                                        putchar(' ');
                                else
                                        putchar('*');
                        }
                        putchar('\n');
                }
        }
        return NULL;
}

void microsleep(int delai)
{
        struct timespec ts = { .tv_sec = delai / 1000000,
                .tv_nsec = 1000 * (delai % 1000000)  };
        nanosleep(&ts, NULL);
}
```

--- * ---

2 Problèmes plus difficiles

Exercice 4 : Le problème du rendez-vous (à N), ou barrière réutilisable

On reprend l'exercice du TP précédent, mais cette fois avec $N > 1$ *threads*.

1. Télécharger, lire, compiler et exécuter le fichier `rendez_vous_a_N_non-sync.c` sur la page du cours. Constater que les *threads* ne se donnent vraiment pas rendez-vous.
2. On veut, comme dans l'exercice précédent, que les *threads* se donnent rendez-vous pour que, pour chaque k, les affichages de l'entier k soient groupés. Par exemple avec $N = 3$, autoriser

   ```
   a : 0
   c : 0
   b : 0
   b : 1
   ```

   mais interdire

   ```
   a : 0
   b : 0
   a : 1
   c : 0
   ```

   Indications :

   — On peut utiliser une variable globale (à manipuler correctement) pour compter, à chaque tour, le nombre de *threads* qui ont terminé de prendre leur temps pour compter puis dans un deuxième temps de compter le nombre de *threads* qui ont terminé d'afficher.

— Le dernier a avoir fini chaque étape a donc un rôle à jouer.
— Avec une structure, on pourra transmettre à chaque *thread* les adresses des objets utilisés pour la synchronisation et leurs indices respectifs.
— On utilisera entre autres deux sémaphores : un qui permet d'attendre que tout le monde ait fini de prendre son temps pour compter avant d'afficher et un autre qui permet d'attendre que tout le monde ait fini d'afficher avant de passer à l'itération suivante.
— Penser à deux portillons : un pour entrer et un pour sortir.

--- * ---

Correction de l'exercice 4 : Cette solution très élégante est due à Allen B. Downey. La globale est protégée par un `mutex`. Un sémaphore d'entrée bloque les *threads* jusqu'à ce que le dernier ait fini de prendre son temps pour compter. Celui-ci ferme la sortie et débloque un thread à l'entrée, qui débloque un thread, ... Finalement, chacun affiche son numéro puis attend à la sortie. Le dernier à atteindre la sortie ferme l'entrée et débloque un thread à la sortie, qui débloque un thread à la sortie, etc.

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5
#define COEFF_ATTENTE 50000

/* attendre delai microsecondes */
void microsleep(int delai);

struct pour_compter {
        int i;
        sem_t *entree;
        sem_t *sortie;
        int *compteur_finis;
        pthread_mutex_t *acces_compteur_finis;
};

void *compteur(void *);
void prendre_du_temps_pour_compter(void);

int main(void)
{
        int i, compteur_finis = 0;
        pthread_t th[N];
        struct pour_compter p_c[N];
        sem_t entree, sortie;
        pthread_mutex_t acces_compteur_finis;

        sem_init(&entree, 0, 0);
        sem_init(&sortie, 0, 1);
        pthread_mutex_init(&acces_compteur_finis, NULL);

        for (i = 0; i < N; ++i) {
                p_c[i] = (struct pour_compter) {
                    .entree = &entree,
                    .sortie = &sortie,
                    .compteur_finis = &compteur_finis,
                    .acces_compteur_finis = &acces_compteur_finis,
                    .i = i
```

```
                };
                if (pthread_create(&th[i], NULL, compteur, &p_c[i]) != 0) {
                        perror("create");
                        return 1;
                }
        }

        for (i = 0; i < N; ++i)
                pthread_join(th[i], NULL);
        sem_destroy(&entree);
        sem_destroy(&sortie);
        pthread_mutex_destroy(&acces_compteur_finis);
        return 0;
}

void *compteur(void *arg)
{
        struct pour_compter *pc = arg;
        int i;
        for (i = 0; i <= 100; ++i) {
                /* Partie concurrente 1 */
                prendre_du_temps_pour_compter();
                /* Fin de partie concurrente 1 */

                pthread_mutex_lock(pc->acces_compteur_finis);
                if (++(*(pc->compteur_finis)) == N) {
                        /* je suis le dernier thread */
                        sem_wait(pc->sortie); /* fermeture du portillon sortie */
                        sem_post(pc->entree); /* ouverture du portillon entrée */
                }
                pthread_mutex_unlock(pc->acces_compteur_finis);

                /* portillon entrée : */
                sem_wait(pc->entree);
                sem_post(pc->entree); /* au suivant ! */

                /* Partie concurrente 2 */
                printf("%s%c : %d\n", i % 2 ? "\033[m" : "\033[31m", 'a'+ pc->i, i);
                /* Fin de partie concurrente 2 */

                pthread_mutex_lock(pc->acces_compteur_finis);
                if (--(*(pc->compteur_finis)) == 0) {
                        /* je suis le dernier thread */
                        sem_wait(pc->entree); /* fermeture du portillon entrée */
                        sem_post(pc->sortie); /* ouverture du portillon sortie */
                }
                pthread_mutex_unlock(pc->acces_compteur_finis);

                /* portillon sortie : */
                sem_wait(pc->sortie);
                sem_post(pc->sortie); /* au suivant */
        }
        return NULL;
}

void prendre_du_temps_pour_compter(void)
{
```

```
        if (rand() % N == 0)
                microsleep((rand() % 10) * COEFF_ATTENTE);
}

void microsleep(int delai)
{
        struct timespec ts = { .tv_sec = delai / 1000000,
                .tv_nsec = 1000 * (delai % 1000000)  };
        nanosleep(&ts, NULL);
}
```

--- * ---

Exercice 5 : Le problème des danseurs (difficile)

Dans ce problème, il y a des danseurs, dont `NB_MENEURS` meneurs et `NB_SUIVEURS` suiveurs. Un meneur ne peut danser qu'avec un suiveur et réciproquement (c'est une danse de salon, par exemple du *rock'n roll*). Dans une boucle infinie :

- Chaque danseur met un certain temps à se préparer.
- Ensuite, il essaie de choisir un partenaire (un suiveur, s'il est meneur et réciproquement) :
  - si aucun partenaire n'est disponible, il ne pourra en choisir un : il attend d'être choisi en s'adossant tristement au mur ;
  - sinon, il choisit (c'est-à-dire ici réveille : on laisse faire le système) un partenaire de l'autre catégorie et l'invite à danser, après lui avoir demandé son nom (ici son numéro) et donné le sien.
- Lorsqu'ils ont choisi ou ont été choisis les danseurs dansent avec leur partenaire.

1. Télécharger et lire le programme `danseurs_non_sync.c` sur moodle.

2. Le compiler et l'exécuter. Noter un certain nombre d'incohérences.

3. Le but est d'arriver à la situation décrite ci-dessus : un danseur choisit ou est choisi (réveille ou est réveillé) par un danseur de l'autre catégorie et danse avec lui.

C'est loin d'être facile, alors voici quelques pistes :

a) les nombres des meneurs et des suiveurs en attente devraient être enregistrés, par exemple dans des variables globales ;

b) à tout instant, il ne peut y avoir qu'un et un seul danseur qui essaie d'en choisir (réveiller) un autre et manipule l'une ou l'autre des deux variables évoquées ci-dessus ;

c) on peut utiliser un sémaphore `file_suiveur` pour les suiveurs qui attendent d'être choisis (réveillés) par un meneur et un sémaphore `file_meneur` pour les meneurs qui attendent d'être choisis (réveillés) par un suiveur ;

d) il faut également faire attention à ce que les partenaires se transmettent correctement leurs noms (numéros).

--- * ---

Correction de l'exercice 5 :

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>

#define NB_MENEURS 5
#define NB_SUIVEURS 6
```

```c
/* attendre delai microsecondes */
void microsleep(int delai);

void *meneur(void *indice);
void *suiveur(void *indice);
void se_preparer();
void suiveur_danser(int num_suiveur, int num_meneur);
void meneur_danser(int num_meneur, int num_suiveur);

int numero_suiveur, numero_meneur;
int nb_meneurs_en_attente, nb_suiveurs_en_attente;
pthread_mutex_t choisir_partenaire;
sem_t file_suiveurs;
sem_t file_meneurs;
sem_t qui_es_tu;

int main(void)
{
        pthread_t meneurs[NB_MENEURS], suiveurs[NB_SUIVEURS];
        int indices_meneurs[NB_MENEURS], indices_suiveurs[NB_SUIVEURS];
        int i;

        pthread_mutex_init(&choisir_partenaire, NULL);
        sem_init(&file_suiveurs, 0, 0);
        sem_init(&file_meneurs, 0, 0);
        sem_init(&qui_es_tu, 0, 0);

        for (i = 0; i < NB_MENEURS; ++i) {
                indices_meneurs[i] = i;
                pthread_create(&meneurs[i], NULL, meneur, &indices_meneurs[i]);
        }
        for (i = 0; i < NB_SUIVEURS; ++i) {
                indices_suiveurs[i] = i;
                pthread_create(&suiveurs[i], NULL, suiveur, &indices_suiveurs[i]);
        }

        for (i = 0; i < NB_MENEURS; ++i)
                pthread_join(meneurs[i], NULL);
        for (i = 0; i < NB_SUIVEURS; ++i)
                pthread_join(suiveurs[i], NULL);
        return 0;
}
void se_preparer()
{
        microsleep(500000 + 400000 * (rand() % 10));
}
void *meneur(void *indice)
{
        int mon_indice = *((int *) indice);
        int indice_partenaire;
        for (;;) {
                se_preparer();
                pthread_mutex_lock(&choisir_partenaire);
                if (nb_suiveurs_en_attente > 0) {
                        --nb_suiveurs_en_attente;
                        numero_meneur = mon_indice;
```

```
                        sem_post(&file_suiveurs);
                        sem_wait(&qui_es_tu);
                        indice_partenaire = numero_suiveur;
                        pthread_mutex_unlock(&choisir_partenaire);
                } else {
                        ++nb_meneurs_en_attente;
                        /* meneur n'a pas pu choisir de partenaire. */
                        pthread_mutex_unlock(&choisir_partenaire);
                        sem_wait(&file_meneurs);
                        numero_meneur = mon_indice;
                        indice_partenaire = numero_suiveur;
                        sem_post(&qui_es_tu);
                }
                meneur_danser(mon_indice, indice_partenaire);
        }
        return NULL;
}
void *suiveur(void *indice)
{
        int mon_indice = *((int *) indice);
        int indice_partenaire;
        for (;;) {
                se_preparer();
                pthread_mutex_lock(&choisir_partenaire);
                if (nb_meneurs_en_attente > 0) {
                        /* suiveur va choisir un partenaire */
                        --nb_meneurs_en_attente;
                        numero_suiveur = mon_indice;
                        sem_post(&file_meneurs);
                        sem_wait(&qui_es_tu);
                        indice_partenaire = numero_meneur;
                        pthread_mutex_unlock(&choisir_partenaire);
                } else {
                        ++nb_suiveurs_en_attente;
                        /* suiveur n'a pas pu choisir de partenaire. */
                        pthread_mutex_unlock(&choisir_partenaire);
                        sem_wait(&file_suiveurs);
                        numero_suiveur = mon_indice;
                        indice_partenaire = numero_meneur;
                        sem_post(&qui_es_tu);
                }
                suiveur_danser(mon_indice, indice_partenaire);
        }
        return NULL;
}

void suiveur_danser(int num_suiveur, int num_meneur)
{
        printf("suiveur %d danse avec meneur %d\n", num_suiveur, num_meneur);
        microsleep(1000000 * 5);
        printf("suiveur %d arrête de danser avec meneur %d\n", num_suiveur, num_meneur);
}
void meneur_danser(int num_meneur, int num_suiveur)
{
        printf("meneur %d danse avec suiveur %d\n", num_meneur, num_suiveur);
        microsleep(1000000 * 5);
        printf("meneur %d arrête de danser avec suiveur %d\n", num_meneur, num_suiveur);
```

```
}

void microsleep(int delai)
{
        struct timespec ts = { .tv_sec = delai / 1000000,
                .tv_nsec = 1000 * (delai % 1000000)  };
        nanosleep(&ts, NULL);
}
```

--- * ---